INF1103 Lab 9: Predictions and Reflections

Name: Haaris Bin Sulaiman Student ID: 2603207

Part 1: Diagnose (Section 1.5)

1. if/elif choosing behaviour	In my Lab 9 inventory_manager.py, create_product() uses if/elif on the product type to decide between PhysicalProduct and DigitalProduct, and add_product() branches on the menu choice (1 or 2) in the same way.
2. Direct creation	PhysicalProduct(...) and DigitalProduct(...) are created directly in both create_product() and add_product(). Adding a new product type would mean editing both.
3. Crash points	float(input("Price: ")), int(input("Stock Quantity: ")), the weight and download size float(input(...)) calls, int(input("New Stock Quantity: ")) in update_stock(), and json.load() on a corrupt inventory.json. A non-number or bad file stops the program with a traceback.

Part 2: Predict First

1. Strategy (Section 2.1): adding a 5% student discount

Question	The business wants a 5% student discount. Without Strategy, which class and method must change? With Strategy?
Prediction	Without Strategy, Order.calculate_total() would need another elif branch for the student discount, so the existing Order class has to be edited. With Strategy, I only add a new StudentDiscount class that implements apply(), and Order does not change.
What happened	I added StudentDiscount to discount_strategy.py and used it with order.set_discount(StudentDiscount()). Order was not modified, and the total came out as $237.50 for a 250 order.
What I changed	Nothing. My prediction matched.

2. Strategy (Section 2.3): BulkDiscount and PercentageDiscount

Question	The order lines add up to 250. What does calculate_total() return with BulkDiscount(200, 20)? Then after set_discount(PercentageDiscount(10))?

Prediction	With BulkDiscount(200, 20), 250 is at or above the 200 threshold, so 20 is taken off and the total is 230. After switching to PercentageDiscount(10), the total is 250 less 10%, which is 225.
What happened	The demo printed No discount (default) $250.00, BulkDiscount(200, 20) $230.00, and set_discount(PercentageDiscount(10)) $225.00. A small order of 25 with BulkDiscount stayed at $25.00 because it was below the threshold.
What I changed	Nothing. My prediction matched.

3. Strategy (Section 2.4): relationship between Order and DiscountStrategy

Question	Is it association, aggregation or composition, and what are the multiplicities?
Prediction	Association. An Order uses a strategy but does not own it, and the same strategy object can be shared by many orders. Multiplicity: Order 0..* to 1 DiscountStrategy.
What happened	My test test_strategy_can_be_shared_by_two_orders passed: two orders shared one PercentageDiscount and gave the same total. The UML shows a plain association line with 0..* on the Order end and 1 on the DiscountStrategy end.
What I changed	Nothing. My prediction matched.

4. Factory (Section 3.2): unknown product type

Question	What happens if I call ProductFactory.create("service", ...)? Which line raises, and what happens if nothing catches it?
Prediction	It reaches the else branch and the raise ValueError(f"Unknown product type: {product_type}") line raises. If nothing catches it, Python prints a traceback ending in ValueError: Unknown product type: service and the program stops.
What happened	In demo_lab9.py the try/except printed Caught ValueError: Unknown product type: service. Running the call without a try/except showed a traceback pointing at the raise line in product_factory.py and then the program ended.
What I changed	Nothing. My prediction matched.

5. Factory (Section 3.3): adding a BundleProduct

Question	How many places must change before the factory and after it? Why does the factory still break OCP, and why is that acceptable?

Prediction	Before the factory, every place that created products had to change (in my code, create_product() and add_product(), plus any loader). After the factory, only one place changes: ProductFactory.create() gets another elif. It still breaks OCP because I edit existing code instead of only adding new code, but it is acceptable because all creation logic is in one small, easy-to-test place instead of being scattered.
What happened	After adding the factory, inventory_manager.py and product_loader.py both call ProductFactory.create(), so the product type logic lives in one place only.
What I changed	Nothing. My prediction matched.

6. Validation (Section 4): DigitalProduct and the negative-stock rule

Question	Product validates that stock cannot be negative. Should DigitalProduct be allowed to skip or change that rule?
Prediction	No. By the Liskov Substitution Principle, a subclass must keep the promises of its parent. If DigitalProduct accepted negative stock, code written for a Product could break when it meets a digital one.
What happened	My test test_both_subclasses_keep_the_same_stock_rule passed: both PhysicalProduct and DigitalProduct returned False for update_stock(-1) and kept their stock, and both constructors raised ValueError for negative stock.
What I changed	Nothing. My prediction matched.

7. Exceptions (Section 5.1): get_positive_float()

Question	A user types abc, then -5, then 12.50. What prints after each entry, and what value is returned?
Prediction	abc prints “Please enter a valid number, e.g. 12.50” (the ValueError from float() is caught). -5 prints “Please enter a number greater than zero.” 12.50 is accepted and the function returns 12.5.
What happened	The output was: Please enter a valid number, e.g. 12.50, then Please enter a number greater than zero., then Returned: 12.5. The loop kept asking until the input was valid.
What I changed	Nothing. My prediction matched.

8. Exceptions (Section 5.2): load_products() with products.csv

Question	How many products load, which messages print and in what order? What happens if the file is renamed: which of except, else and finally run?

Prediction	Three products load (P001, D001, P003). The service row is skipped for an unknown type and the abc price row is skipped for a bad float. Then 3 products loaded. prints, and Load attempt finished. prints last. If the file is renamed, except runs (file not found), else is skipped so no “products loaded” line, and finally still runs.
What happened	The output showed both Skipping row lines (service: Unknown product type, then the abc row: could not convert string to float), then 3 products loaded., then Load attempt finished. For missing_products.csv it printed Products file not found: missing_products.csv and then Load attempt finished.
What I changed	Nothing. My prediction matched.

Part 3: Automated Tests (Section 6)

I wrote 22 unit tests in the tests/ folder (discounts, factory, validation, input helper and CSV loading) and ran them in Docker with python -m unittest discover -s tests -t . -v. The result was Ran 22 tests ... OK (see lab9_tests.png). The tests are written so that they could be run automatically after every push, which is what a CI pipeline would do.

Part 4: Reflection (optional, Section 8)

Strategy and Factory both came from the same idea: stop hard-coding choices inside one class. Before this lab, my Order and my inventory_manager.py would have needed new if/elif branches for every new discount or product type. Now a new discount is a new class and a new product type is one extra branch in the factory, so the rest of the program stays untouched.

I also learned the difference between validation and exception handling. Validation (if + raise ValueError in Product and OrderLine) refuses bad data at the moment it would be created, while try/except (in get_positive_float, load_products and load_inventory) decides what the program does when something goes wrong, such as re-asking the user or skipping a bad CSV row. The CSV example showed why finally matters: “Load attempt finished.” printed whether the file existed or not.

The automated tests changed how I check my work. Earlier I only looked at the screen output, but the tests catch mistakes without me re-typing inputs. One thing I found tricky was keeping update_stock() returning False while the constructor raises ValueError, or understanding why the factory still breaks OCP. Next time I would probably write the tests first scripts first.